

FACULDADE ENGENHEIRO SALVADOR ARENA

Lucas Junqueira Gonçalves – 082230029

Murilo Umbelino dos Santos – 082230013

Victor Mendes de Andrade Ferreira – 082230015

Guilherme Alves Barbosa – 082220014

Arquitetura de Software

Cripto ULA

Vinicius Borges

05/2026

São Bernardo do Campo

Sumário

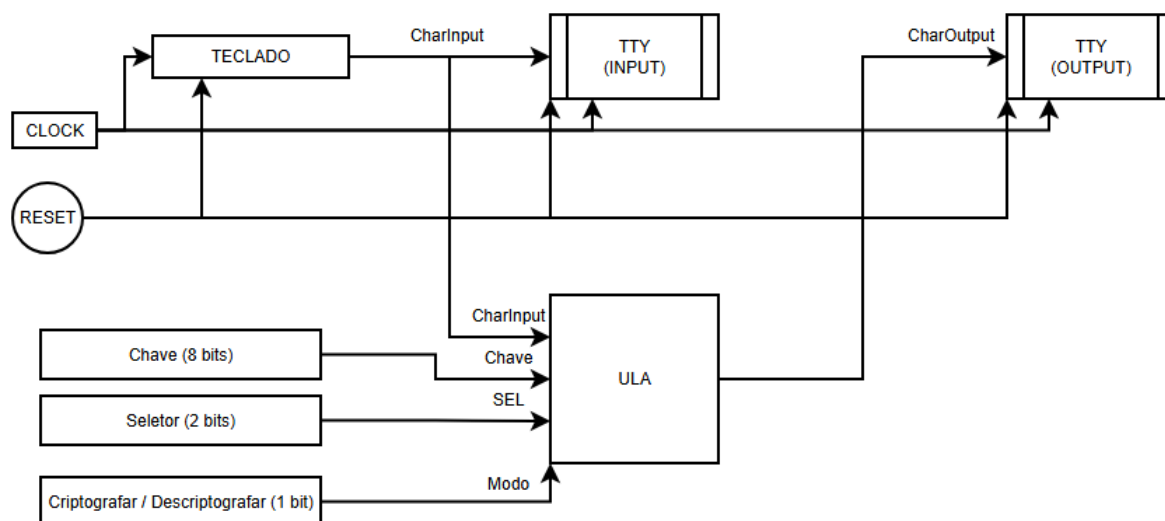
Sumário.....	1
Introdução	2
Diagrama de Blocos.....	3
Descrição do Bloco de Processamento.....	3
Módulos do Sistema	3
Entrada (Keyboard)	4
Núcleo Criptográfico.....	4
Controle de Modo.....	5
Saída (TTY).....	6
Explicação Detalhada dos Algoritmos.....	6
Operação XOR (SEL = 00)	6
Cifra de César Binária (SEL = 01)	6
Inversor de Bits / NOT (SEL = 10).....	7
Permutação de Bits (SEL = 11)	7
Conclusão e Discussões Finais	7
Melhorias Futuras.....	8

Introdução

A documentação a seguir remete as informações sobre o projeto de uma Unidade Lógica e Aritmética Criptográfica (Cripto-ULA) desenvolvida no ambiente da ferramenta chamada Logisim, um software que possibilita a modelagem de fluxos e a criação de circuitos com portas lógicas e operações aritméticas com bits e barramentos. O sistema foi criado utilizando os princípios fundamentais dos conteúdos da matéria de Arquitetura de Computadores e da criptografia digital através da implementação de quatro algoritmos distintos: a Cifra de César Binária, a operação lógica XOR, a Inversão de bits e a Permutação estrutural de bits. A arquitetura do projeto segue uma abordagem modular hierárquica, onde cada componente possui responsabilidades bem definidas e interfaces claras de comunicação através de subcircuitos dedicados.

O objetivo principal do sistema é processar caracteres ASCII de 8 bits gerados em tempo real por um dispositivo de teclado (*Keyboard*), aplicando transformações criptográficas baseadas na seleção do usuário entre modos de criptografia (*Encrypt*) e descryptografia (*Decrypt*), e exibindo o resultado em um terminal de texto (*TTY*).

Diagrama de Blocos



Descrição do Bloco de Processamento

O bloco de processamento é responsável por realizar a criptografia e a descriptografia de cada caractere. O bloco é dividido em três partes principais: os módulos que transformam os dados, a unidade que escolhe qual cálculo usar e o pino que define o sentido da operação (se vai codificar ou decodificar).

Abaixo está o fluxo sequencial que o dado para a saída criptografada ou descriptografada:

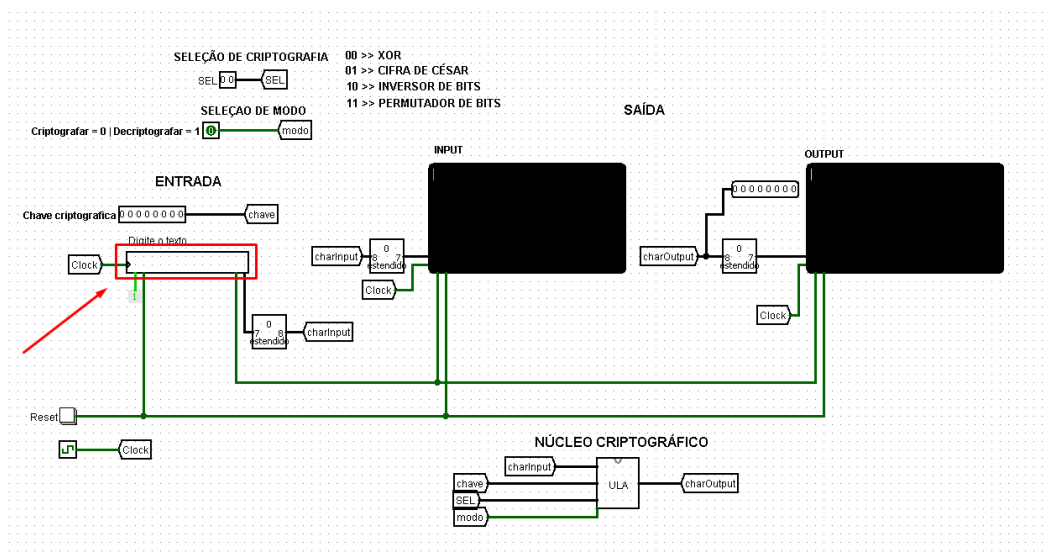
1. O caractere em formato ASCII (8 bits) chega na entrada do bloco.
2. Esse mesmo dado é espalhado simultaneamente para os quatro módulos de criptografia, de modo que todos calculam suas respectivas transformações ao mesmo tempo.
3. As quatro saídas geradas chegam até as portas de entrada de um Multiplexador (MUX).
4. O MUX, controlado pelo sinal do seletor, filtra essas respostas e deixa passar apenas o resultado do algoritmo que o usuário escolheu.
5. O dado selecionado é enviado para a saída do bloco, seguindo em direção ao terminal.

Módulos do Sistema

O circuito pode ser dividido quatro blocos que controlam o fluxo da informação:

Entrada (Keyboard)

O dispositivo de entrada do sistema é o componente Keyboard nativo do Logisim. Ele simula um teclado comum e armazena temporariamente os caracteres digitados em um buffer interno. Cada tecla pressionada gera um valor binário de 8 bits correspondente à codificação ASCII daquele caractere. O caractere é exibido em um display TTY para mostrar a entrada do usuário e é levado ao núcleo criptográfico para realizar a criptografia ou descriptografia.



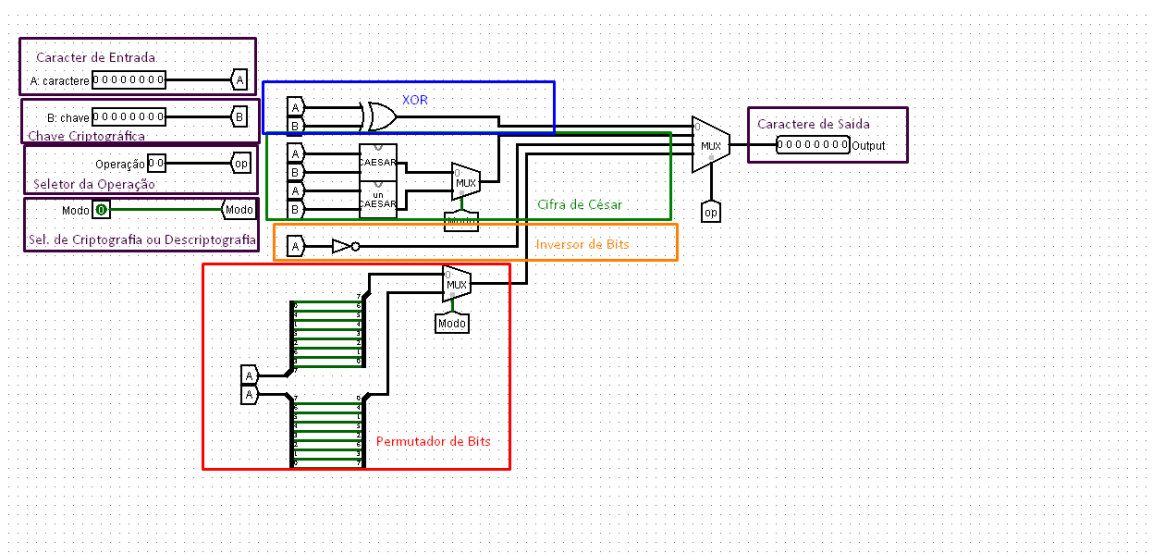
Núcleo Criptográfico

O Núcleo Criptográfico é composto pela ULA (Unidade Lógica e Aritmética) do projeto. Este módulo engloba todos os subcircuitos responsáveis pelos cálculos matemáticos e lógicos.

Ele recebe as informações repassadas e selecionadas no circuito “Main”, sendo: O caractere vindo do teclado (“Keyboard”), a chave configurada pelo usuário (“Chave”) e os sinais de controle, modo e seletor criptográfico. Como o circuito é modularizado, o núcleo criptográfico funciona de forma isolada, recebendo os fios de entrada e entregando apenas a resposta final já multiplexada.

O seletor de criptografia é utilizado para definir a operação escolhida pelo usuário. O seu dado é utilizado no MUX final do processo de conversão para definir qual das entradas serão realmente utilizadas para a saída final da conversão. Ele é controlado por uma chave de 2 bits definida como “SEL” na main e depois definida como “op” na ULA. Segue a tabela de utilização da chave SEL para definição da criptografia:

SEL [1:0]	Algoritmo
00	XOR
01	Cifra de César
10	Inversor de Bits
11	Permutador de Bits

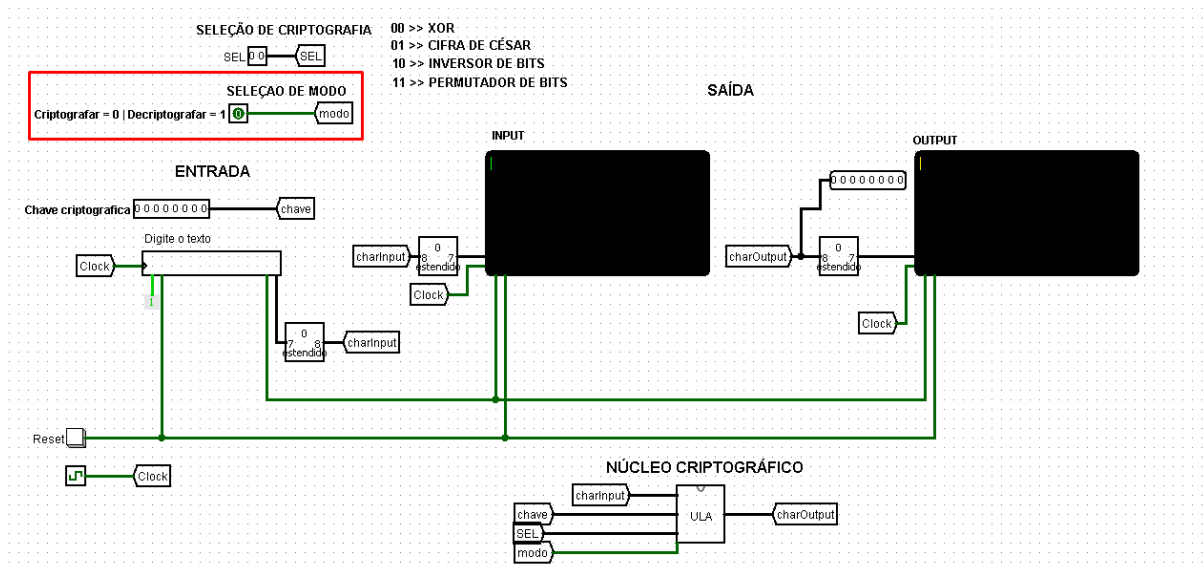


De forma geral, a ULA (Núcleo Criptográfico) recebe todos os dados definidos na main e realiza a conversão do caractere de acordo com a operação selecionada. A chave “op” define no MUX final qual das entradas será utilizada para a saída.

Controle de Modo

Esse módulo é controlado por uma chave estática de 1 bit (Modo) no circuito principal, que dita se o sistema vai funcionar no modo ENCRYPT (0) ou DECRYPT (1). A função dele é mudar o comportamento interno da ULA.

Nos algoritmos que não são idênticos na ida e na volta (como o César e a Permutação), esse sinal atua diretamente desviando o fluxo dos dados. No caso do César, ele define se o circuito vai somar ou subtrair, e na permutação, ele chaveia entre o embaralhamento normal e a fiação inversa.



Saída (TTY)

O componente TTY é o terminal de texto que exibe os resultados na tela. Ele funciona de forma síncrona com o teclado: toda vez que um caractere é processado pela ULA, o circuito de pulso ativa o pino Write do TTY para imprimir a letra na tela.

Um detalhe importante da arquitetura é que o TTY do Logisim trabalha com o padrão ASCII de 7 bits. Por isso, a saída de 8 bits da ULA passa por um extensor de bits (Bit Extender) configurado para reduzir o barramento, descartando o bit mais significativo para que o caractere seja compatível com o display.

Explicação Detalhada dos Algoritmos

Abaixo está o detalhamento de como cada um dos quatro algoritmos foi construído no Logisim e como eles se comportam matematicamente:

Operação XOR (SEL = 00)

O algoritmo XOR realiza uma operação lógica bit a bit entre o caractere digitado e uma chave de 8 bits definida por interruptores. Esta é a operação que retorna verdadeiro (1) quando as entradas são diferentes e retorna falso (0) para quando elas são iguais. A grande vantagem do XOR na eletrônica digital é que ele é uma operação involutiva, ou seja, aplicar a operação duas vezes com a mesma chave faz o dado voltar ao estado original. Por causa dessa propriedade matemática, não precisamos de circuitos separados para codificar e decodificar.

Cifra de César Binária (SEL = 01)

A Cifra de César Binária trabalha com a tabela ASCII inteira fazendo aritmética de 8 bits. Para a criptografia é realizada a adição da chave em cada caractere e para a descryptografia é realizada a subtração da chave em cada caractere:

- **No modo ENCRYPT:** O circuito direciona o caractere e a chave para um componente somador (Adder) de 8 bits.
- **No modo DECRYPT:** O sinal de controle vira o fluxo para um componente subtrator (Subtractor) de 8 bits, que faz a operação inversa (subtrai a chave do caractere criptografado).

Inversor de Bits / NOT (SEL = 10)

Esse módulo aplica o complemento de 1 bit a bit no caractere usando portas inversoras (NOT), portanto, o que é 0 vira 1 e o que é 1 vira 0. Assim como o XOR, o NOT também desfaz a si mesmo se for aplicado duas vezes, eliminando a necessidade de circuitos de ida e volta separados.

Um detalhe desse algoritmo é que ele gera comportamentos diferentes para letras maiúsculas e minúsculas no TTY. Como as minúsculas começam com os bits 011 na tabela ASCII, quando aplicado o NOT e o dado é cortado para 7 bits no TTY, o resultado cai na faixa dos caracteres de controle (valores de 0 a 31), fazendo a tela ficar em branco (invisível). Já as maiúsculas começam com 010, fazendo o resultado após o NOT cair na faixa dos caracteres visíveis (como o número "8", símbolos etc.).

Permutação de Bits (SEL = 11)

A permutação não altera os bits e nem realiza somas ou subtrações, ela apenas muda os bits de posição física usando uma fiação cruzada. Para fazer isso no Logisim, foram utilizados os componentes chamados "Splitters".

- **No bloco de ENCRYPT:** O primeiro splitter separa o barramento de 8 bits em fios individuais e os conecta em um segundo splitter seguindo a ordem: [b7 b6 b5 b4 b3 b2 b1 b0] → [b0 b2 b4 b6 b1 b3 b5 b7]
- **No bloco de DECRYPT:** Para a descryptografia os fios foram mapeados ao contrário (por exemplo, o pino 0 da esquerda vai para o pino 7 da direita, o pino 4 vai para o 6) desfazendo o cruzamento anterior e recuperando o caractere original. Um MUX local dentro do bloco seleciona qual das duas malhas de fios será usada com base na chave CryptDecrypt.

Conclusão e Discussões Finais

O desenvolvimento deste projeto permitiu consolidar de forma prática e visual os conceitos fundamentais de Arquitetura de Computadores e Sistemas Digitais. A criação da Unidade Lógica e Aritmética Criptográfica (Cripto-ULA) no Logisim de forma hierárquica provou-se altamente eficiente, já que dividir o sistema em blocos menores (subcircuitos) facilitou o processo de desenvolvimento, a organização das trilhas de dados e, principalmente, a depuração de erros.

Melhorias Futuras

Embora o sistema atual cumpra com rigor todos os requisitos propostos, a arquitetura modular projetada deixa permitir expansões e otimizações em cenários futuros:

- **Gerenciamento Dinâmico de Chaves:** Em vez de utilizar interruptores estáticos no painel para definir as chaves de criptografia (do XOR e de César), o circuito poderia ser expandido com registradores internos de 8 bits para permitir que o usuário digitasse e salvasse uma nova chave dinamicamente pelo próprio teclado.
- **Compatibilidade Total de Barramento (8 bits nativos):** Para mitigar o problema das letras que ficam invisíveis no TTY (geração de caracteres de controle nos modos NOT e Permutação), o display de texto poderia ser substituído por uma matriz de LEDs customizada ou por uma tela que opere com um padrão estendido de 8 bits, garantindo que qualquer símbolo gerado pela ULA seja visualizado graficamente.